

Trabajo Integrador de Programación.

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas.

Materia: Programación.

Alumno: Julián García.

Fecha : 17/06/2026

Tabla de contenido

Objetivos del proyecto	1
Marco teórico	2
Decisiones técnicas y arquitectura De software.	3
Diagrama de flujo	3
Evidencias de ejecución del sistema Control de errores.	4
Dificultades técnicas afrontadas Y resolución.	5
Biografía/webgrafía.	5

Objetivo del Proyecto

El trabajo integrador tiene como objetivo diseñar e implementar un sistema de software interactivo en Python que automatice la administración, filtrado y análisis estadístico de una base de datos geográfica. El desarrollo está enfocado en garantizar la persistencia de la información mediante archivos CSV y robustecer la integridad lógica del sistema a través de rutinas estrictas de validación de datos por teclado. En principio el trabajo se comenzó en grupo de 2 personas, pero por inconvenientes personales del otro integrantes cerca de la fecha de entrega del proyecto, se tuvo que continuar de forma individual, modalidad que se encuentra autorizada en las consignas de la plataforma.

Marco teórico.

El desarrollo de sistemas de software orientados a la gestión de datos requiere una comprensión sólida de las estructuras de almacenamiento, el flujo de control y las metodologías de persistencia. En el ámbito del lenguaje Python 3.x, los conceptos aplicados en este Trabajo Práctico Integrador se fundamentan en los siguientes pilares de las ciencias de la computación:

1. Estructuras de Datos Lineales: Listas

Las listas en Python son colecciones de elementos ordenadas, mutables y dinámicas. A nivel de arquitectura de software, actúan como el contenedor principal en memoria (`lista_paises`) para almacenar el conjunto de registros. Su propiedad de mutabilidad permite la inserción dinámica de nuevos elementos mediante métodos como `.append()`, así como la modificación directa de sus índices durante los procesos de actualización de datos.

2. Estructuras de Datos Clave-Valor: Diccionarios

Cada registro geográfico individual del dataset se modela mediante un diccionario. A diferencia de las secuencias indexadas numéricamente, los diccionarios permiten asociar datos mediante pares de claves (strings) y valores (tipos de datos mixtos: `int`, `string`). Esta estructura optimiza la legibilidad del código al representar entidades del mundo real con atributos semánticos claros: `'nombre'`, `'poblacion'`, `'superficie'` y `'continente'`.

3. Modularización y Programación Funcional

La modularización consiste en fragmentar un sistema complejo en componentes independientes llamados funciones, donde cada una responde al principio de responsabilidad única (una función = una tarea específica). Las funciones reciben parámetros de entrada, ejecutan un bloque lógico aislado y retornan opcionalmente un valor, facilitando la mantenibilidad, reutilización de código y la depuración de errores.

4. Flujo de Control y Estructuras Repetitivas

La lógica del sistema está gobernada por estructuras condicionales (`if-elif-else`) para la toma de decisiones y la bifurcación del menú principal. En paralelo, las estructuras repetitivas como los bucles determinados (`for`) permiten iterar y filtrar las colecciones de datos, mientras que los bucles indeterminados (`while True`) garantizan la persistencia de la interfaz de consola y el aislamiento de los ciclos de validación de entradas de usuario.

5. Algoritmos de Ordenamiento y Filtrado

El ordenamiento es la operación de organizar los elementos de una lista bajo un criterio de colación específico (alfabético o numérico) de forma ascendente o descendente. En Python, se implementa mediante funciones integradas como `sorted()` o el método `.sort()`, parametrizados con expresiones `lambda` para evaluar claves específicas dentro de los diccionarios. El filtrado consiste en la extracción selectiva de elementos que satisfacen un predicado lógico booleano (rangos de valores o coincidencias de cadenas).

6. Persistencia de Datos y Archivos CSV

Para que la información sobreviva al cierre de la ejecución del programa en la memoria RAM, se requiere persistencia en almacenamiento secundario. Los archivos CSV (Comma-Separated Values) constituyen un estándar de texto plano ampliamente utilizado para almacenar datos tabulares. Python proporciona el módulo nativo `csv` para automatizar la lectura (`csv.DictReader`) y la escritura secuencial de filas, interactuando de forma transparente con las estructuras de listas y diccionarios del software.

Decisiones técnicas y arquitectura de software.

Se decidió utilizar una lista de diccionarios como estructura de datos principal (lista_paises). Cada país representa una entidad independiente con claves tipificadas de forma estricta :

.nombre que se usa title() para guardar los nombres con primera letra en mayúscula por una cuestión estética de alfabética, luego población y superficie con integer positivo en los cuales se valida en la entrada que el número ingresado sea positivo y entero y por último continente donde se decidió utilizar un catalogo cargado con los continentes ya existentes.

El programa se diseñó bajo el paradigma de programación funcional, donde cada opción del menú delega la responsabilidad a una función específica (agregar_pais, actualizar_datos etc) esto permite aislar los bloques de código y facilita el mantenimiento del software.

Se implemento el módulo nativo csv para automatizar la sincronización de datos. Al iniciar el programa, la función lee el archivo países.csv y reconstruye la estructura en la memoria RAM. Al seleccionar la opción de salida, el sistema reescribe el archivo físico garantizando la persistencia de datos.

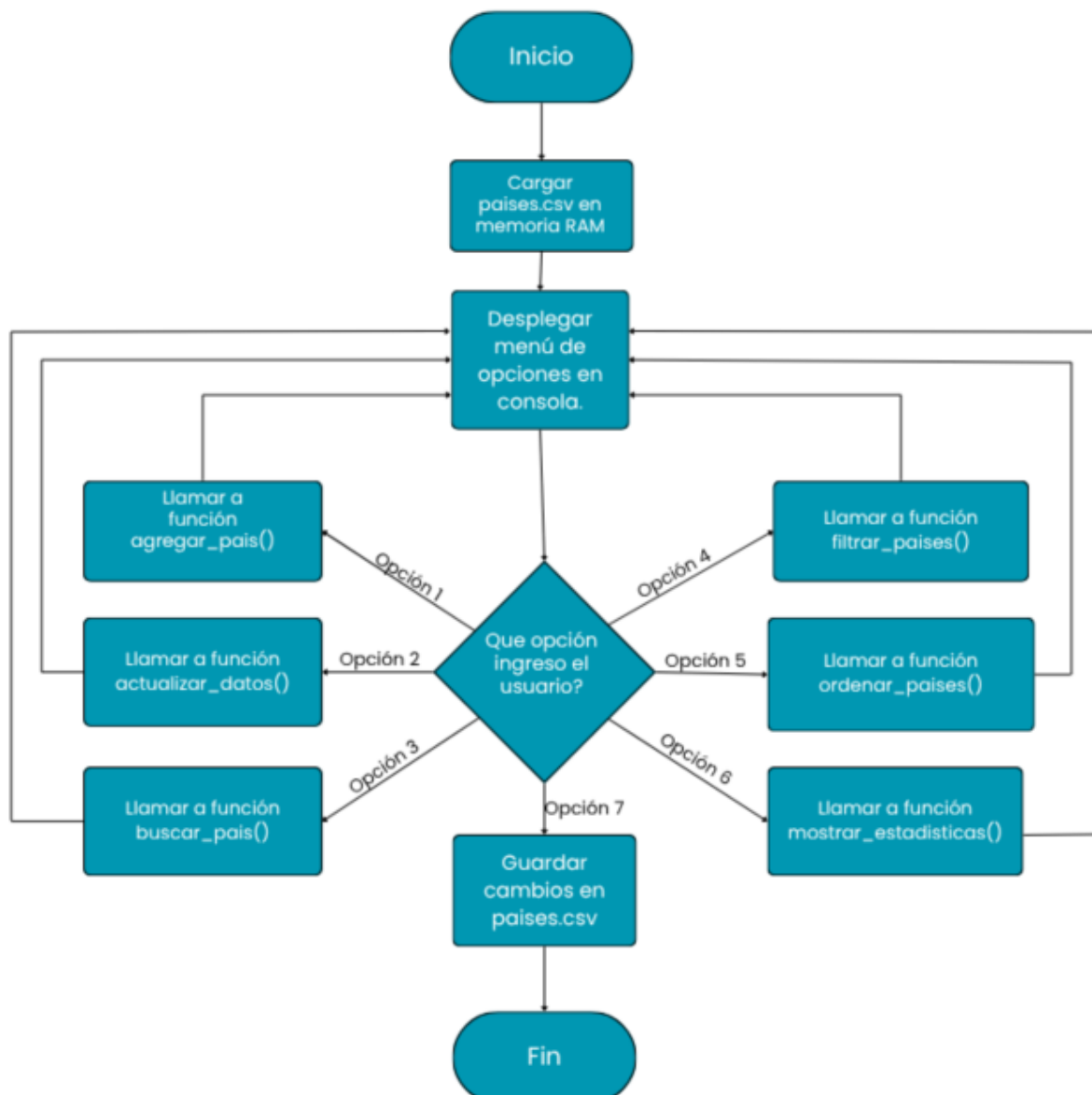
Diagrama de flujo del programa.

Figura 1: Diagrama de flujo.

Evidencias de Ejecución del Sistema y Control de Errores.

<pre> 315 lista_paises = cargar_datos_csv(ruta) 316 317 while True: 318 print("\n=====") 319 print(" Gestión de datos de países. ") 320 print("=====") 321 print("1. Agregar país") 322 print("2. Actualizar población/superficie") 323 print("3. Buscar país por nombre") </pre> PROBLEMAS 83 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SPELL CHECKER 79 PS C:\Users\Julían\Documents\UTN\Programación> & C:/Users/Julían/AppData/Local/Prog <pre> ===== Gestión de datos de países. ===== 1. Agregar país 2. Actualizar población/superficie 3. Buscar país por nombre 4. Filtrar países 5. Ordenar países 6. Mostrar estadísticas 7. Guardar y Salir Seleccione una opción: </pre> <i>Figura 2: Interfaz de usuario en consola y despliegue del menú principal de opciones tras la lectura exitosa del dataset CSV.</i>	Seleccione una opción: 1 --- Registrar Nuevo País --- Nombre del país: ecuador4 El nombre del país debe contener únicamente letras. Nombre del país: ecuador Población (entero positivo): s Ingrese un número entero válido. Población (entero positivo): 10000000 Superficie en km² (entero positivo): 250000 Seleccione el continente: 1. América 2. Asia 3. Europa 4. Oceanía 5. África 6. Antártida Seleccione una opción (1-6): 1 ¡Ecuador registrado con éxito! <i>Figura 3: Módulo de alta interactiva ejecutando los ciclos de validación contra ingresos alfabéticos erróneos y normalización de texto (.title()).</i>
Seleccione una opción: 4 --- Filtrar países --- 1. Por Continente 2. Por Rango de Población 3. Por Rango de Superficie Seleccione opción de filtrado: 1 Seleccione el continente a filtrar: 1. América 2. Asia 3. Europa 4. Oceanía 5. África 6. Antártida Seleccione una opción (1-6): █ <i>Figura 4: Interfaz del submódulo de filtrado ejecutando la selección paramétrica por catálogo numérico de continentes.</i>	Seleccione una opción: 6 --- Estadísticas de dataset --- Mayor Población: India (1408000000 hab.) Menor Población: Ecuador (10000000 hab.) Promedio de Población: 198091772.73 habitantes Promedio de Superficie: 3274482.09 km² Cantidad de países por continente: - América: 4 - Asia: 2 - Europa: 2 - Oceanía: 1 - África: 2 <i>Figura 5: Reporte analítico de estadísticas descriptivas descriptas por consola y confirmación de persistencia física en el archivo paises.csv.</i>
<pre> ===== Gestión de datos de países. ===== 1. Agregar país 2. Actualizar población/superficie 3. Buscar país por nombre 4. Filtrar países 5. Ordenar países 6. Mostrar estadísticas 7. Guardar y Salir Seleccione una opción: 7 </pre> Guardando cambios en 'paises.csv'... <i>Figura 6: Guardado de datos en archivo csv al salir del programa.</i>	

Dificultades Técnicas Afrontadas y Resolución.

Durante la etapa de control y pruebas funcionales del software se identificaron vulnerabilidades y errores lógicos que fueron solucionados de forma metódica. Una de las decisiones de diseño más críticas consistió en restringir el campo del continente mediante un catálogo cerrado de opciones válidas, obligando al usuario a ingresar exclusivamente un continente existente.

Respecto a la validación de los nombres de los países, por razones lógicas de alcance no se utilizó la misma estrategia restrictiva. Validar la existencia geopolítica real de una nación hubiese requerido recursos ajenos a las consignas fijadas por la cátedra, tales como el consumo de una API externa o el almacenamiento estático de una lista con todos los países del mundo dentro del código fuente, lo cual no se consideraba óptimo ni eficiente para esta entrega. En su lugar, se robusteció la integridad sintáctica asegurando que el ingreso contenga únicamente caracteres alfabéticos y espacios.

Otro obstáculo técnico detectado en las primeras ejecuciones del sistema fue la imposibilidad de cargar el dataset debido a fallas de ruta. Esta dificultad se resolvió mediante la correcta ubicación del archivo países.csv en el directorio raíz del script principal, garantizando una lectura transparente al iniciar el flujo.

Conclusiones del Proyecto

Como conclusión del proyecto, este trabajo integrador constituyó una experiencia fundamental para asimilar de forma práctica los contenidos conceptuales dictados en el primer cuatrimestre de programación. La teoría y las prácticas de estructuras de datos cobraron un sentido de utilidad real en un entorno simulado de gestión comercial.

Finalmente, se destaca que la calidad de un software no reside únicamente en cumplir con el flujo ideal o feliz del programa, sino en su robustez para gestionar todos los posibles errores del operador. El desarrollo fortaleció la capacidad de capturar de forma preventiva cualquier falla de ingreso por teclado para proteger la integridad lógica del programa y la persistencia del almacenamiento secundario.

Bibliografía /webgrafía

Python Software Foundation. (2026). *Python 3.14 Documentation: Built-in Types (Sets, Lists, Dictionaries)*. Documentación oficial del lenguaje. **Enlace:** python.org

GitHub Docs. (2026). *About READMEs and managing repositories*. Documentación oficial de GitHub para la gestión de proyectos y control de versiones. **Enlace:** github.com

Cátedra de Programación I - UTN. (2026). Universidad Tecnológica Nacional.
Enlace: utn.edu.ar

Píldoras Informáticas. (2023). *Manejo de Archivos y Estructuras de Datos Tabulares (CSV) en Python*. Canal Académico de Programación y Ciencias de la Computación.
Enlace: https://www.youtube.com/watch?v=LYdsJ88e_ag

Wikipedia. (2026). *Anexo: Países y territorios dependientes por población*. Tabla estadística global de habitantes de las naciones del mundo. **Enlace:** wikipedia.org

Wikipedia. (2026). *Anexo: Países por superficie*. Registro geográfico oficial de extensiones territoriales en kilómetros cuadrados (km²). **Enlace:** wikipedia.org

Enlace a Video explicativo en Youtube. <https://youtu.be/yIbFB448VNw>

Enlace a repositorio en GitHub. <https://github.com/Exodusjcb/trabajo-integrador-programacion>